Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

- **Regression:** The goal is to predict a **continuous numerical value**.
  - *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.
- **Classification:** The goal is to predict a **discrete category or class label**.
  - *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

- **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

In [1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
```

```
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg` , `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error` . This metric tells us the average of the squared differences between the predicted and actual values.

In [2]:
```
import numpy as np
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# lr_model = ...

lr_model = LinearRegression()

# 2. Train the model
# lr_model.fit(...)

lr_model.fit(X_train_reg, y_train_reg)

# 3. Make predictions
# y_pred_reg = lr_model.predict(...)

y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f} dollars.")
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01 dollars.
```

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

- **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

In [3]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:

1. Create an instance of the `LogisticRegression` model.
2. Train the model using the classification training data.
3. Make predictions on the test data.
4. Evaluate the model using `accuracy_score`.

In [4]:
```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# log_model = ...
log_model = LogisticRegression()

# 2. Train the model
# log_model.fit(...)
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
# y_pred_cls = log_model.predict(...)
y_pred_cls = log_model.predict(X_test_cls)


# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**
2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**
3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

1. Regression vs. Classification

- Regression problems are problems that predict a continuous numeric target, like price, temp, or minutes.
- Its like asking yourself "How much or How many" of something.
- When it comes to classification, It's like compartmentalizing something into different buckets or discrete categories while answering questions like (which one? Yes or No?)

In [5]:
```
print("Coefficients:", lr_model.coef_)
print("Intercept:", lr_model.intercept_)
```

```
Coefficients: [ -0.43695889 -38.29728935]
Intercept: 133.30726668128784
```

2. Model Output: Coefficients and Intercept

After running the printed code we are given the coefficients and the intercept.

Coefficients

Age

- For the coefficients We have age which can be rounded to 0.44(it's also negative).
- So this is an indication that for each additional year of age lowers the predicted fair by about 0.44.

Pclass

- For Pclass we have -38.30. This has more of a significant effect.
- Each one step increases in class number starting from one to two to three, and this drops the predicted fair by around $38.30.
- First class is expected to be pricier but as the classes go up It gets cheaper with third class being the cheapest.

- So as the class goes up, the fare should go down, with one being the most expensive and going up to three being the lowest fare.

Intercept

- The second value that we're given is the intercept, which could also be considered as the baseline.
- So the predicted fares for the model when age is zero when p class is zero.
- So, in simpler words, this is just where the math line starts

3. Why MSE for Regression but Accuracy for Classification

- We know that MSE measures distance. This type of measurement is needed for a number that is never exactly (not a fixed value).
- Accuracy counts right vs wrong, and that metric wouldn't be right for fares since its continuous.